

Addressing Concept Drift in IoT Anomaly Detection: Drift Detection, Interpretation, and Adaptation

Lijuan Xu , Ziyu Han , Dawei Zhao , Xin Li , Fuqiang Yu , and Chuan Chen 

Abstract—Anomaly detection plays a vital role as a crucial security measure for edge devices in Artificial Intelligence and Internet of Things (AIoT). With the rapid development of IoT (Internet of Things), changes in system configurations and the introduction of new devices can lead to significant alterations in device relationships and data flows within the IoT, thereby triggering concept drift. Previously trained anomaly detection models fail to adapt to the changed distribution of streaming data, resulting in a high number of false positive events. This paper aims to address the issue of concept drift in IoT anomaly detection by proposing a comprehensive Concept Drift Detection, Interpretation, and Adaptation framework (CDDIA). We focus on accurately capturing the concept drift of normal data in unsupervised scenarios. To interpret drift samples, we integrate a search optimization algorithm and the SHAP method, providing a comprehensive interpretation of drift samples at both the sample and feature levels. Simultaneously, by utilizing the sample-level interpretation results for filtering new and old samples, we retrain the anomaly detection model to mitigate the impact of concept drift and reduce the false positive rate. This integrated strategy demonstrates significant advantages in maintaining model stability and reliability. The experimental results indicate that our method outperforms five baseline methods in adaptability across three datasets and provides interpretability for samples experiencing concept drift.

Index Terms—Concept drift, interpretability, anomaly detection, IoT.

Manuscript received 30 January 2024; revised 3 March 2024; accepted 1 April 2024. Date of publication 26 April 2024; date of current version 10 December 2024. This work was supported in part the National Key R&D Program of China under Grant 2023YFB3107303, in part by the Natural Science Foundation of Shandong Province under Grant ZR2021MF132 and Grant ZR2020YQ06, in part by the National Natural Science Foundation of China under Grant 62172244, in part by the Young innovation team of colleges and universities in Shandong province under Grant 2021KJ001, in part by the Innovation Ability Promotion Project for Small and Medium-sized Technology-based Enterprise of Shandong Province under Grant 2022TSGC2098 and Grant 2023TSGC0150, in part by the Pilot Project for Integrated Innovation of Science, Education and Industry of Qilu University of Technology (Shandong Academy of Sciences) under Grant 2022JBZ01-01, in part by Taishan Scholars Program under Grant tsqn202211210, in part by the “20 New Universities” Project of Jinan City under Grant 202333045, and in part by the Pilot Project for the Integration of Science, Education and Industry of Qilu University of Technology (Shandong Academy of Sciences) under Grant 2023PX100 and Grant 2023RCKY145. Recommended for acceptance by Associate Editors: Z. Guo, H. Xia, Y. Wang, and R. Chouchane. (Corresponding authors: Dawei Zhao; Xin Li.)

The authors are with the Key Laboratory of Computing Power Network and Information Security, Ministry of Education, Shandong Computer Science Center (National Supercomputer Center in Jinan), Shandong Provincial Key Laboratory of Computer Networks, Shandong Fundamental Research Center for Computer Science, Qilu University of Technology (Shandong Academy of Sciences), Jinan 250014, China (e-mail: xulj@sdsas.org; zhaodw@sdsas.org; lixin@sdsas.org).

Digital Object Identifier 10.1109/TSUSC.2024.3386667

I. INTRODUCTION

WITH the formulation of carbon neutrality goals, there is a growing demand for sustainability in the industrial sector year after year. In this process, the widespread application of Artificial Intelligence and Internet of Things (AIoT) technologies across various domains has brought numerous benefits to society. Among these, anomaly detection algorithms are applied in areas such as the Internet of Things (IoT) [1], [2], particularly gaining widespread use in industrial control systems [3].

Exceptionally, deep learning-based anomaly detection algorithms have made significant progress in terms of detection accuracy and reducing false positive rates. These algorithms prove effective in detecting attacks and potential threats to IoT, enabling timely preventive measures [4], [5], [6].

However, machine learning-based detection algorithms often operate under the assumption of a closed-world, where training and testing samples are independent and identically distributed. In the realm of IoT security, this assumption frequently does not hold true. In practical IoT applications, there exist extensive streams of non-stationary data from on-site sensors and actuator states. Over time, changes in the configuration of IoT edge devices or adjustments in industrial control system processes contribute to the common occurrence of concept drift. For instance, events like equipment cleaning leading to increased water flow and speed in industrial water treatment systems or switching primary and backup valves can trigger instances of concept drift. The emergence of concept drift may adversely impact the performance and stability of detection models.

In recent years, various methods have been proposed to address the issue of concept drift, including:

- *Data Monitoring and Drift Detection*: Timely detection of concept drift is the first step in addressing the problem. Monitoring statistical characteristics of input data, such as feature distributions and label distributions, as well as monitoring changes in model prediction results and performance, can be achieved using statistical methods, hypothesis testing, cumulative errors, and other techniques. Once a change in data distribution is detected, corresponding measures can be taken.
- *Dynamic Model Updates*: Dynamically update models to adapt to new data as the data distribution changes. Incremental learning or online learning methods can be employed to introduce new data and update model parameters while preserving old knowledge to reflect the new data distribution.

- *Periodic Retraining*: Periodically retraining models is a simple yet effective method to address concept drift. Regularly retraining models using the latest data helps maintain the model's adaptability to new data.

The IoT is highly interconnected, with strong correlations among various device components. Changes in system configurations and the introduction of new devices can lead to significant alterations in device relationships and data flows within the IoT, resulting in concept drift. However, identifying which specific changes in device components trigger the occurrence of concept drift remains a challenge. Unfortunately, most current concept drift techniques do not focus on and explain the specific device components causing concept drift.

Therefore, the development of concept drift detection and adaptation methods with interpretability becomes crucial. Such research not only contributes to enhancing the reliability of anomaly detection but also provides more effective safeguards for the maintainability and security of IoT.

We have developed a framework named CDDIA to address the concept drift issue in IoT anomaly detection, covering detection, interpretation, and adaptation components. Our contributions can be summarized in the following three points:

- We have developed a framework named CDDIA to address the concept drift issue in IoT anomaly detection. CDDIA specifically focuses on detecting concept drift in normal data within unsupervised anomaly detection, tackling the challenge of lacking label information in real-world scenarios. It provides an accurate method for capturing normal data distribution changes in unsupervised scenarios.
- We propose a comprehensive interpretability approach to address the common lack of interpretability of current concept drift techniques. Utilizing a search optimization algorithm is employed to identify new samples where concept drift occurs and outdated old samples. Combining this with the SHAP (Shapley Additive exPlanations) method, we precisely identify the device components responsible for concept drift. This innovation provides a comprehensive interpretation for drift samples at both the sample and feature levels.
- CDDIA can filter out new samples experiencing concept drift and old samples that are not outdated. The impact of concept drift is alleviated by retraining the anomaly detection model. This comprehensive adaptation strategy exhibits significant advantages in maintaining model stability and reliability. We validated the generality and effectiveness of CDDIA by comparing it with five baseline methods on three widely accepted datasets (SWaT, BATADAL, and CICIDS).

II. RELATED WORK

Concept Drift was initially proposed by Schlimmer and Granger [7], who believed that changes in the target environment mainly cause the drift of the target concept. Widmer et al. formally defined Concept Drift in literature [8], stating that it results from changes in the contextual environment within the data stream, leading to variations in the target concept. In

simple terms, Concept Drift refers to the unpredictable changes in the underlying feature distribution of a data stream over time. Currently, scholars have focused on three main aspects of Concept Drift research [9], [10]: Concept Drift Detection [11], the interpretability of Concept Drift [12], and Concept Drift Adaptation [13].

A. Concept Drift Detection

Drift detection methods employ proactive mechanisms to monitor drift and trigger warnings or other mechanisms to make necessary adjustments to the model. This direction primarily includes Concept Drift detection methods based on error rates, window-based methods, and data distribution-based methods.

1) *Error Rate Based Concept Drift Detection*: Concept Drift detection methods based on error rates are widely used and involve monitoring the error rate changes of online base learners to identify Concept Drift. Among them, DDM proposed by Gama et al. [14] effectively detects abrupt Concept Drift by maintaining the error rate of base classifiers and utilizing mean and variance of error rates as statistics. However, this method is less sensitive to gradual Concept Drift and relies heavily on classifier performance. EDDM [15] is an early Concept Drift detection method similar to DDM. However, it uses distance error rates instead of classifier error rates, making it more suitable for both abrupt and gradual Concept Drift with higher accuracy. The ECDD algorithm by Ross et al. [16] uses an exponentially weighted moving average detection method and is particularly suitable for gradual Concept Drift. RDDM [17] and its variant FW-DDM [18] enhance sensitivity to gradual Concept Drift based on DDM. HDDM [19] employs the Hoeffding inequality for Concept Drift detection.

2) *Window-Based Concept Drift Detection*: Window-based Concept Drift detection methods typically use two data instance windows, one for storing old instances and the other for storing new instances in the data stream. Comparing the differences between these two window instances reflects changes in data distribution and triggers a drift signal. The window size can be fixed or adaptive. The adaptive sliding window algorithm (ADWIN) proposed by Bifet et al. [20] adjusts the window size based on data distribution, and drift alarms occur when the average distribution difference between two consecutive windows is more significant than a predefined threshold. ADWIN performs well, limiting false positives and false negatives, but only applies to one-dimensional data, requiring additional time and memory for multidimensional data. ADWIN2 is an improved version with lower time and memory requirements. Another implicit Concept Drift detector is the Outlier Detection for Concept Drift (OCDD), proposed by Can et al. [21], based on a sliding window mechanism and real-time calculation of the percentage of outlier values to determine Concept Drift. Pair Learners (PL), proposed by Bach et al. [22], is a window-based drift detection method using two base learners: stable and reactive. The stable learner is trained on previous instances and predicts new instances. In contrast, the reactive learner learns recent instances and predicts the current window's instances, detecting Concept Drift by comparing their accuracy differences.

3) *Data Distribution-Based Concept Drift Detection*: Data distribution-based Concept Drift detection algorithms detect drift by comparing old (or historical) data with current data instances. These methods are often combined with window-based methods, analyzing statistical significance to calculate data distribution changes and obtaining information about where the drift occurred. The Hybrid Forest algorithm focuses on using Random Forest with Hoeffding Trees, determining Concept Drift occurrence through multiple stages of combination. The Information Theory Approach (ITA) by Dasu et al. [23] uses *kdqTree* to partition old and new data into bins, calculates the distinguishability of density using KL divergence, and finally uses hypothesis testing to determine Concept Drift.

B. Concept Drift Interpretation

Understanding the impact of Concept Drift on a model and its subsequent performance decline is facilitated through the interpretation of Concept Drift. The explanatory results encompass information about “when,” “how,” and “where.” Specifically, “when” refers to when the target concept experiences drift, “how” signifies the severity of Concept Drift, and “where” indicates the specific features where the drift occurs. These status details serve as both the output of Concept Drift detection algorithms and inputs for Concept Drift adaptation methods.

CADE [24] is a Concept Drift detection and interpretation method for secure applications based on contrastive learning. It provides a reasonable unsupervised Concept Drift sample interpretation method by perturbing inputs and observing changes in the nearest centroid in latent space. OWAD [25] is another framework for Concept Drift detection and interpretation tailored for secure applications. It identifies crucial samples causing Concept Drift, reducing the cost of labeling drift samples. Panda et al. [26] employ classifier gradients to comprehend the contribution of various features to the disparity in posterior distributions between training and testing, thereby identifying features correlated with drift.

C. Concept Drift Adaptation

Concept Drift adaptation methods include model retraining, ensemble-based drift adaptation methods, and adjustments to existing models.

1) *Model Retraining*: Model retraining involves retraining the model to adapt to new data. Bach et al. [22] proposed a drift adaptation method based on pair learners, maintaining two learning models. One model predicts based on all data, while the other predicts based on recent data. If the first model frequently makes erroneous predictions, it is replaced by the second one.

2) *Ensemble-Based Concept Drift Adaptation*: Ensemble-based Concept Drift adaptation methods use a set of base classifiers to form an ensemble. Bifet et al. proposed a Bagging-based method [27] that retrains new base classifiers with recent samples, replacing the poorest-performing base classifier and then using the remaining base classifiers for prediction. Chu [28] introduced a Boosting-based method that, assuming non-drift data classification errors follow a Gaussian distribution, uses hypothesis testing to monitor prediction accuracy for handling

Concept Drift. Gomes [29] proposed an adaptive random forest algorithm that employs resampling and adaptive methods to handle different types of Concept Drift, adjusting the random forest based on drift detection results.

3) *Existing Model Adjustment Methods*: Existing model adjustment methods are designed for models capable of adaptive learning, exhibiting partial self-updating capabilities when the underlying data distribution changes. Hulten [30] introduced an ultra-fast decision tree learner that trains alternative subtrees with the latest arriving data and replaces the original decision tree during Concept Drift.

III. PRELIMINARIES

A. Preparation

To conduct the study, data collected during different time periods, denoted as X^o for past samples, represents old data. X^n represents new data samples collected within a certain time span compared to old data. Both X^o and X^n undergo normalization. The dataset is chronologically divided, where a portion of the old sample X^o is used as the training set X_{train}^o , and another portion serves as the test set X_{test}^o .

The training set X_{train}^o is employed to train an unsupervised deep learning-based anomaly detection model denoted as f , with AutoEncoder used as the anomaly detection model in this paper. The training parameters of model f are saved for subsequent use. The symbols are labeled as shown in Table I.

IV. PROPOSED APPROACH

The proposed method initiates by training an anomaly detection model on a historical dataset containing only normal data. The output results of the anomaly detection model are calibrated to transform probability values into more accurate probability estimates, enforcing meaningful probability information in the model output. Statistical comparison of the distributions of new and old samples is performed through hypothesis testing, utilizing KL divergence to measure the difference between the two probability distributions.

A search optimization algorithm is employed to identify new samples indicating concept drift and outdated old samples. SHAP [31] (SHapley Additive exPlanations) is utilized to explain the new samples exhibiting concept drift, aiding in identifying the feature dimensions contributing to concept drift. Finally, the new samples indicating concept drift and the non-outdated old samples are combined to retrain the anomaly detection model, adapting to the detected drift. Fig. 1 illustrates the framework of the proposed method.

A. Confidence Calibration

Confidence calibration is a well-explored technique in machine learning and deep learning [25]. This technique aims to ensure that the model’s probability predictions align with actual observed outcomes. As the model’s predicted results may not perfectly match the ground truth, confidence calibration aims to enhance the understanding of the model’s prediction reliability and provide more meaningful probability distributions for

TABLE I
NOTATIONS

Notation	Description
x^o, X^o	The old data collected in the past
x^n, X^n	Newly collected data (data that may have experienced concept drift)
S_n, A_n	The nth sensor or actuator component.
f	Unsupervised anomaly detection model (AutoEncoder)
$C(\cdot)$	calibrator
$\mathbb{H}_K(\cdot)$	Calibrating the output using K bins frequency histogram to compute the discrete distribution
P_{org}, Q_{org}	Discrete distribution (histogram) of old or new samples
m^o, m^n	Mask vectors for X^o and X^n , indicating whether the i -th sample in X^o and X^n is selected

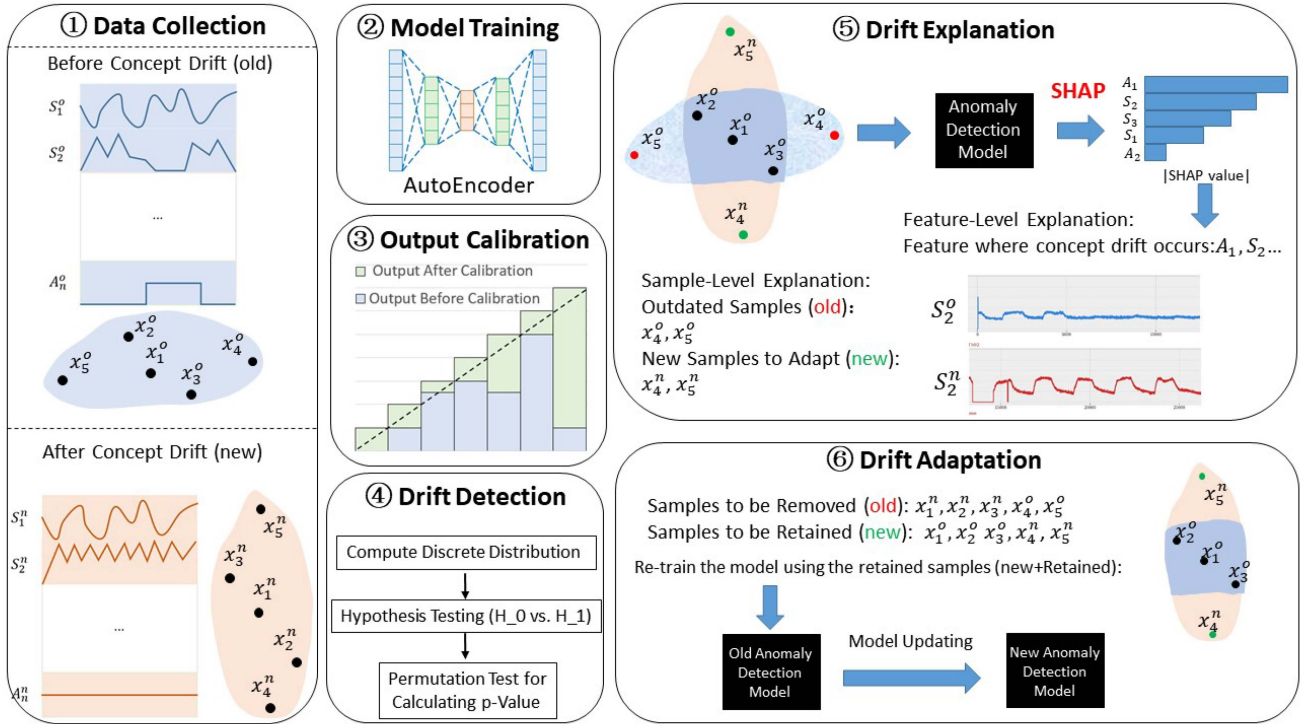


Fig. 1. Overall framework of DeepINT.

subsequent decisions. Through calibration, we can intuitively understand that a prediction calibrated to 0.8 has an expected accuracy of 80. Confidence calibration essentially involves finding a mapping function (denoted as $C(\cdot)$) that maps the raw output to the desired confidence. However, existing research primarily focuses on calibrating supervised classifiers, requiring fully labeled two-class samples to calculate true probabilities. In the context of anomaly detection with only normal samples, this approach is not applicable. Therefore, we need an unsupervised calibration method that does not rely on prior knowledge of anomalies.

First, the testing set X_{test}^o is evaluated using the trained anomaly detection model f , and the root mean square error (RMSE) is computed as the output of the anomaly detection model f . Second, the uncalibrated outputs are sorted in descending order. The uncalibrated output refers to the numerical value directly output by the anomaly detection model, specifically

the reconstruction error (root mean square error, RMSE) of the autoencoder. The sorted uncalibrated output values are used as inputs x_{group} for the calibrator C , and the uncalibrated output values are the outputs y_{group} generated by C . The generation of y_{group} involves calculating the normalized value within the range $[0, 1]$ for each uncalibrated output value based on its position in the sorted list relative to all data points. This way, y_{group} reflects the relative position of each uncalibrated probability value in the sorted list. This process is represented by (1) and (2)

$$x_{group} = f(X_{test}^o) \quad (1)$$

$$y_{group} = C(x_{group}). \quad (2)$$

In (1) and (2), $f(\cdot)$ is the anomaly detection model, and $C(\cdot)$ is the calibrator. Next, we use a monotonic regression method, specifically the Isotonic Regression method, to fit the confidence calibration curve. In this approach, x_{group} and y_{group} serve as

training data to fit and calibrate the anomaly detection model. To ensure that the output of the fitted model conforms more closely to a probability distribution, we set the upper and lower bounds of the fitted function to 1 and 0, respectively.

B. Drift Detection

When the phenomenon of concept drift occurs, detecting it becomes a crucial problem. Initially, we calculate the calibrated outputs of the testing set X_{test}^o from the old samples X^o and the new samples X^n . To represent the distributions, the discrete distributions of $C(f(x^o))$ and $C(f(x^n))$ are computed using K bins frequency histograms, resulting in the old distribution P_{org} and the new distribution Q_{org} as shown in (3) and (4)

$$P_{org} \leftarrow \mathbb{H}_K(C(f(X^o))) \quad (3)$$

$$Q_{org} \leftarrow \mathbb{H}_K(C(f(X^n))). \quad (4)$$

In (3) and (4), $f(\cdot)$ is the anomaly detection model, $C(\cdot)$ is the calibrator, and $\mathbb{H}_K(\cdot)$ computes the discrete distribution of the calibrated output using a K bins frequency histogram. P_{org} represents the discrete distribution (histogram) of the old samples, and Q_{org} represents the discrete distribution (histogram) of the new samples.

Specifically, the dataset is divided into K intervals, each containing approximately the same number of samples. Then, the number of samples in each interval is counted, and it is plotted as a frequency histogram. This process transforms continuous data into discrete probability distributions, which are used for subsequent hypothesis testing.

Next, hypothesis testing is employed to statistically compare these two discrete distributions P_{org} and Q_{org} using a permutation test. The specific steps are as follows:

Null and Alternative Hypotheses: Null Hypothesis H_0 : $C(f(X^o))$ and $C(f(X^n))$ are drawn from the same distribution, indicating no concept drift. Alternative Hypothesis H_1 : $C(f(X^o))$ and $C(f(X^n))$ are drawn from different distributions, indicating concept drift.

Test Statistic and KL Divergence: The KL divergence is used to measure the difference between the two discrete distributions P_{org} and Q_{org} and is defined as

$$D_{KL}(P||Q) = \sum_{i=1}^n P(i) \log \left(\frac{P(i)}{Q(i)} \right). \quad (5)$$

The original test statistic S_{org} is calculated using the KL divergence between P_{org} and Q_{org}

$$S_{org} \leftarrow D_{KL}(P_{org}||Q_{org}). \quad (6)$$

Permutation Test for p-value: Permutation testing assumes that samples are exchangeable under the null hypothesis. Samples are randomly permuted, and the difference values are calculated after each permutation. The p-value is then calculated by comparing the observed test statistic S_{org} with the distribution of test statistics obtained from all permutations

$$p \leftarrow \frac{1 + \sum_{i=1}^{N_p} 1[S_{org} \leq D_{KL}(P_i^R||Q_i^R)]}{N_p + 1}. \quad (7)$$

Algorithm 1: Drift Detection and Interpretation in CDDIA.

Input: $x^o \in X_N^o, x^n \in X^n, \mu$
Output: Samples of concept drift

- 1 $P_{org} \leftarrow \mathbb{H}_K(C(f(X^o))), Q_{org} \leftarrow \mathbb{H}_K(C(f(X^n)))$
- 2 $D_{KL}(P||Q) = \sum_{i=1}^n P(i) \log \left(\frac{P(i)}{Q(i)} \right)$
- 3 $S_{org} \leftarrow D_{KL}(P_{org}||Q_{org})$
- 4 $p \leftarrow \frac{1 + \sum_{i=1}^{N_p} 1[S_{org} \leq D_{KL}(P_i^R||Q_i^R)]}{N_p + 1}$
- 5 **if** $p < \mu$ **then**
 - 6 $m^o \leftarrow \text{random_uniform}(\text{shape_of}(X^o), 0, 1)$
 $m^n \leftarrow \text{random_uniform}(\text{shape_of}(X^n), 0, 1)$
 Update m^o and m^n
 $m_{bin}^o \leftarrow \text{binarize_with_threshold}(m^o, t)$
 $m_{bin}^n \leftarrow \text{binarize_with_threshold}(m^n, t)$
 $X_{old_delete}^o \leftarrow X^o[m_{bin}^o == 0]$
 $X_{old_remain}^o \leftarrow X^o[m_{bin}^o == 1]$
 $X_{new_shift}^n \leftarrow X^n[m_{bin}^n == 1]$
- 7 **end**
- 8 **return** $p, X_{old_delete}^o, X_{old_remain}^o, X_{new_shift}^n$

In (7), N_p is the number of permutations, and $1[S_{org} \leq D_{KL}(P_i^R||Q_i^R)]$ is an indicator function.

In permutation testing, the p-value measures the significance of the difference between the two discrete probability distributions. A smaller p-value indicates a more significant difference, making it more likely to reject the null hypothesis. Typically, p-values range between 0 and 1, and a comparison with a statistical threshold μ determines the existence of a significant difference. If the p-value is less than μ , the null hypothesis is rejected, suggesting concept drift. Otherwise, if the p-value is greater than or equal to μ , the null hypothesis is not rejected, indicating no significant difference and no concept drift.

C. Drift Interpretation

1) Sample-Level Interpretation: If concept drift occurs, an optimization problem is formulated to identify the crucial samples causing the drift in normal data. The masking approach is introduced, where masking vectors m^o and m^n correspond to X^o and X^n , and m_i^o and m_i^n indicate whether the i th sample in X^o and X^n is selected. If $m_i^o/m_i^n = 1$, the i th sample in X^o and X^n is selected; if $m_i^o/m_i^n = 0$, the i th sample in X^o and X^n is not selected. The specific implementation process includes:

First, Create empty tensors m^o and m^n with the same shape as the old samples X^o and new samples X^n . Then, Initialize m^o and m^n with uniform distribution in the range $[0, 1]$. Optimize loss functions L_1 and L_2 as shown in (8) and (9)

$$L_1 = D_{KL} \left\{ \mathbb{H}_M(C(f(x^n))) \parallel \mathbb{H}_M((m^o \odot C(f(X^o))) \oplus (m^n \odot C(f(X^n)))) \right\} \quad (8)$$

$$L_2 = \mathbb{E}_{m \in m^o \oplus m^n} [m \log m + (1 - m) \log (1 - m)]. \quad (9)$$

In (8) and (9), the symbol $\oplus$ concatenates two row vectors element-wise, forming a new row vector. The symbol $\odot$

denotes the Hadamard product (element-wise product) of two row vectors.

L_1 represents the accuracy loss, reconstructing the new distribution to “repair” the drift in normal data. KL divergence is used to measure the distance between the new distribution and the reconstructed distribution. The relative frequency bin number is denoted as M , and $\mathbb{H}_M(\cdot)$ converts the calibrated output into an M -bin relative frequency histogram vector. L_2 is the cross-entropy loss, measuring the determinacy level of the mask (either close to 0 or close to 1). Use stochastic gradient descent (SGD) optimizer to optimize the above loss functions and obtain masking vectors m^o and m^n . Binarize the masking vectors m^o and m^n using a threshold t . Values greater than t are set to 1, and values less than or equal to t are set to 0. This way, the discretized masking vectors m^o and m^n have only two values: 0 and 1. According to the masks, classify the corresponding samples into three parts: old samples that are outdated (old_delete), old samples that are not outdated (old_remain), and new samples where concept drift occurs (new_shift).

- $m_i^o = 0$ corresponds to the i th old sample x_i^o classified as outdated. (class1)
- $m_i^o = 1$ corresponds to the i th old sample x_i^o classified as not outdated. (class 2)
- $m_i^n = 1$ corresponds to the i th new sample x_i^n classified as experiencing concept drift. (class 3)

2) *Feature-Level Interpretation*: Feature-level interpretation builds upon sample-level interpretation. In this paper, we utilize SHAP to provide feature-level interpretation for detected drifting samples. In the aforementioned sample-level interpretation, we categorize samples corresponding to different masking vectors into three classes. We employ one class of samples to create a SHAP explainer. The SHAP explainer utilizes this dataset to analyze the structure and weights of the anomaly detection model, calculating the contributions of features to the prediction results. By using one class of samples for SHAP creation, we not only reduce computational costs and time but also more accurately capture and evaluate the contributions of features to drifting samples. Using the three classes of samples as the test dataset for computing SHAP values, the output Shapely values are utilized for drifting feature importance assessment. Shapely values offer insights into the contribution of each feature to the prediction results of the anomaly detection model. Through the analysis of Shapely values, we can identify which features play a crucial role in the occurrence of concept drift in data samples. Thus, facilitating the assessment of feature importance.

D. Adaptation to Drift

Utilizing sample-level interpretation, we select two classes of samples (non-outdated old samples) and three classes of samples (new samples experiencing concept drift) as the new training dataset. By retraining the anomaly detection model, we prevent the model from forgetting situations where there are no outdated samples in the old distribution while also learning from the new distribution where concept drift occurs in normal samples. Experimental results demonstrate that the method of retraining the model with new samples obtained through sample-level



Fig. 2. Layout of SWaT testbed [32].

interpretation has a significant advantage. This approach effectively reduces the model’s false positive rate, thereby enhancing the stability and reliability of the model.

V. DATASETS

In the following experiments, three datasets were employed, namely SWaT [32], BATADAL [33], and CICIDS2017 [34]. It is essential to note that WADI [35] was not utilized for concept drift experiments as it only collected data for a concentrated period of 16 days in 2017. Therefore, WADI was not included in this study.

A. SWaT

SWaT (Secure Water Treatment) is an experimental setup representing a scaled-down version of large-scale modern water treatment systems in the real world. The layout of the SWaT Testbed is shown in Fig. 2.

The dataset from SWaT has multiple versions [36], [37], and for the experiments in this paper, data collected during different time periods in 2015 and 2017 were utilized. The data collection process in 2015 spanned 11 days, with the system operating 24 hours a day. During the initial seven days of this period, there were typically no attacks or faults. In the last four days, attack tests were conducted, comprising a total of 36 attacks. The attacked points included actuators (such as valves, pumps, etc.) and sensors (such as water level sensors, flow meters, etc.). In 2017, data were collected for six days with continuous operation for 136 hours, including network traffic and historical data, without any attack tests.

B. BATADAL

The BATADAL (Battle of the Attack Detection Algorithms) dataset is generated using the epanetCPA 56 MATLAB toolkit, simulating hydraulic changes in a water treatment system when subjected to cyber-physical attacks. This dataset comprises 43 features, including 31 sensors and 12 actuators. It has a long duration of data collection, being generated over the course

of a year. The dataset, released in 2016, consists entirely of normal operations, without any attacks. Subsequently, in 2017, a dataset spanning three months was released, including multiple instances of attacks.

C. CICIDS2017

The CICIDS2017 (Canadian Institute for Cybersecurity Intrusion Detection Systems 2017) dataset is designed for research in network intrusion detection. The flow data in the CICIDS2017 dataset is derived from a simulated network environment, emulating various real-world network activities and attacks. The dataset encompasses different types of network traffic, including normal network traffic and various types of network attacks such as DoS (Denial of Service), DDoS (Distributed Denial of Service), and malicious software. As different types of attacks are launched during distinct time periods to create the CICIDS2017 dataset, the attack patterns in the dataset evolve over time, resulting in multiple instances of concept drift within the CICIDS2017 dataset.

VI. EXPERIMENTS

A. Experiment Setup and Preparation:

We conducted our experiments on Google Colab. The experiments were carried out using Python version 3.10 and PyTorch version 2.1.0+cu118.

We obtained two sets of data from the SWaT dataset: 1. 495,000 continuous records were collected during the uninterrupted normal operation of SWaT in December 2015 (excluding the initial 30 minutes, which was maintenance and deemed irrelevant for training). 2. 13,661 historical records were collected during the continuous normal operation of the SWaT system for 136 hours in June 2017.

Data Preprocessing: All data samples underwent Min-Max Normalization for preprocessing. It is essential to note that this dataset exclusively comprises normal data, and no anomaly data is present.

Dataset Partitioning: The dataset with 51 features from 2015 was divided into two segments: The first 100,000 records for training the anomaly detection model. The next 100,000 records for the concept drift calibration test set.

All data from 2017 was used as the test set for concept drift detection, interpretation, and adaptation.

Configuration Parameters: Iterations for permutation tests were set to 1,000. The number of bins (K) in the distribution histogram was set to 10. The significance threshold (μ) for drift detection was set to 0.01 (i.e., $p < 0.01$ indicates a significant difference). For drift interpretation, the threshold (t) for binarization was set to 0.5. The number of bins (M) for calculating relative frequencies, and $\mathbb{H}_M(\cdot)$ converting the calibrated output to an M-bin relative frequency histogram vector, was set to 50.

1) Performance Metrics: In the assessment of the adaptive performance of our method to concept drift, we utilize the False Positive Rate (FPR), defined as follows:

$$FPR = \frac{FP}{FP + TN}. \quad (10)$$

Here, FP represents the quantity of False Positives, and TN represents the quantity of True Negatives. The False Positive Rate is the proportion of normal samples incorrectly classified as anomalies by the anomaly detection model.

2) Baseline Method:

- *SRP (Streaming Random Patches)* [38] is an ensemble method tailored for streaming classification, combining random subspaces and online bagging.
- *Hoeffding Tree* [30] is an incremental, anytime decision tree induction algorithm capable of learning from massive data streams, assuming the distribution generating the instances remains constant over time. Hoeffding Trees leverages the fact that a small sample is often sufficient to choose the best splitting attribute. This idea is mathematically supported by the Hoeffding bound, quantifying the number of observations needed to estimate certain statistics within a specified accuracy (in our case, attribute goodness) range.
- *BOLE (Boosting-like Online Learning Ensemble)* [39] is a Boosting-like Online Learning Ensemble based on Adaptable Diversity-based Online Boosting (ADOB). This method aims to enhance the accuracy of ensemble learning by relaxing the requirements of expert voting and modifying the internal concept drift detection method.
- *ADWIN Bagging* [40] is an online bagging method incorporating the ADWIN algorithm as a change detector. Upon detecting concept drift, the worst-performing member in the ensemble is replaced with a new (empty) classifier based on ADWIN's error estimate.
- *OWAD* [25] is a framework for concept drift detection and interpretation in security applications. It identifies important samples causing concept drift, reducing the cost of labeling drifting samples. OWAD achieves this by limiting the update of important model parameters to avoid forgetting the old effective distribution while supporting the update of non-important parameters to generalize to the new distribution and adapt to concept drift.

B. Feature-Level Interpretation

To delve into the feature-level intricacies of detected concept drift, we leverage SHAP, an interpretability tool. The approach unfolds in two key phases.

Initiating the process, we craft a SHAP interpreter utilizing samples from class 1, representing outdated old samples. This strategic selection enables us to scrutinize the model's architecture and weights. The interpreter meticulously computes the contribution of each feature to the model's prediction results. Leveraging class 1 samples optimizes computational efficiency and temporal considerations while enhancing the precision in understanding and evaluating the contribution of features in the context of drift. Subsequently, we pivot to class 3 samples, encompassing new instances undergoing concept drift, as our test dataset for SHAP value computation. The resultant Shapley values, derived from SHAP analysis, become instrumental in evaluating the importance of features. This step facilitates the discernment of which features play pivotal roles in causing concept drift within the dataset. Table II briefly describes the

TABLE II
DEVICE TYPE AND DESCRIPTION

Name	Description	Type
AITx0y	Analysar Indicator Transmitter	Sensor
DPITx0y	Differential Pressure Indicator Transmitter	Sensor
FITx0y	Flow Indicator Transmitter	Sensor
LITx0y	Level Indicator Transmitter	Sensor
PITx0y	Pressure Indicator Transmitter	Sensor
MVx0y	Motorised Valve	Actuator
Px0y	Pump	Actuator

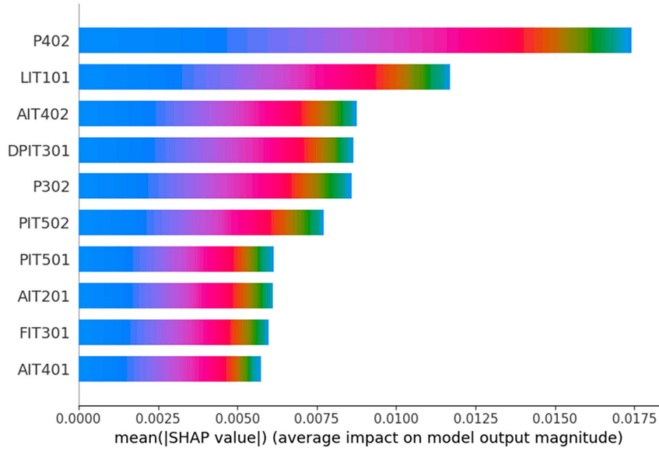


Fig. 3. Explain the samples filtered by CDDIA that experience concept drift, with the ranking of components of devices where concept drift occurs as shown in the figure.

devices as well as the types, and Table III describes selected device component functions.

The visualization in Fig. 3 unveils the feature importance ranking. On the x -axis, importance values, calculated through SHAP, quantify the impact of each feature on the model's predictions. Larger values signify more substantial influence. The y -axis showcases feature names. Notably, features positioned higher on the chart hold greater significance in contributing to concept drift. Scrutinizing Fig. 3, the discerned features contributing to concept drift, arranged by descending importance, are P402, LIT101, AIT402, DPIT301, P302, and PIT502 (In this paper, we observe the first six features.).

Fig. 4(a) to (g) depict the numerical variations of selected device components in the SWaT dataset collected in 2015 and 2017. The Dividing Line is utilized to separate the two time periods (2015 and 2017). It can be observed that concept drift occurs in certain device components when collected after a two-year interval.

Fig. 4(a) and (b) depict the actuators P402 and P401. It is evident that in 2015, P402 was active while P401, serving as a backup component, remained inactive. In 2017, the roles were reversed, with P401 being active and P402 remaining inactive. The interchangeability of these actuators as backup components introduces concept drift, leading to false positives in anomaly detection due to the model learning only one configuration in the past.

- Fig. 4(c) represents the sensor AIT402. The variation trend of this sensor differs between the two years.
- Fig. 4(d) shows the sensor DPIT301. The average values collected by this sensor vary between the years, with approximately 20 in 2015 and around 18 in 2017.
- Fig. 4(e) and (f) depict the actuators P302 and P301, echoing the situation discussed for P402 and P401 in Fig. 4(a) and (b).
- Fig. 4(g) illustrates the sensor PIT502. The trend of data collected by this sensor varies between the two years, with smoother changes in 2015 and more regular fluctuations in 2017.

C. Concept Drift Adaptation

Concept drift adaptation experiments were conducted separately on three datasets (SWaT, BATADAL, and CICDIS2017). Interesting observations were made during the experiments. On the SWaT dataset, some classical concept drift detection and adaptation methods showed a false positive rate of 0. This might be attributed to the fact that the test set consisted of normal operational data collected in 2017 by the iTrust team. However, interpretable experiments revealed that concept drift did occur in the SWaT dataset in 2017. Moreover, when disabling the concept drift detection and adaptation modules of these methods, the false positive rate remained at 0. This experimentation indicates that these classical methods cannot effectively detect the concept drift that occurred in the SWaT dataset.

The experimental results are presented in Table IV. On the SWaT dataset, our method reduced the false positive rate from 17% (No-Update) to within 3%. On the BATADAL dataset, our method almost reduced the false positive rate to 0, with only one false positive. In addition to experiments on physical datasets from two industrial control systems, we also used a traffic dataset, CICIDS2017. On this dataset, our method outperformed other baseline methods in terms of false positive rates.

D. Ablation Experiment

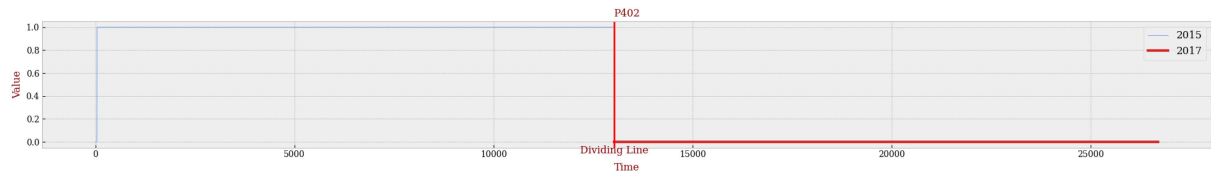
In addition to the baseline methods introduced earlier, we also set up two additional baseline methods for conducting ablation experiments:

- No-Update: Refers to the original anomaly detection model without any drift remediation measures.
- Retrain: Involves retraining the model with both old and new samples combined.

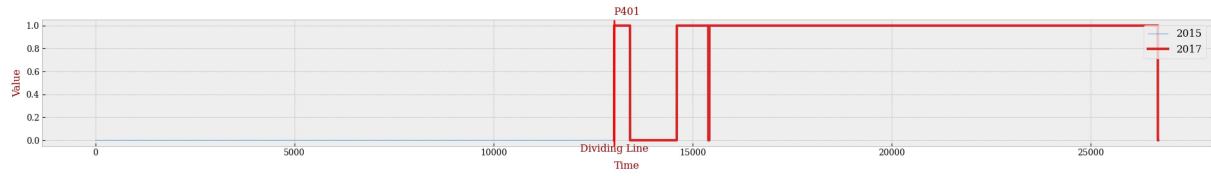
As shown in Fig. 5, our method exhibits a significant reduction in false positive rates compared to the original model without any drift remediation measures and the model retrained from scratch.

E. Time Efficiency

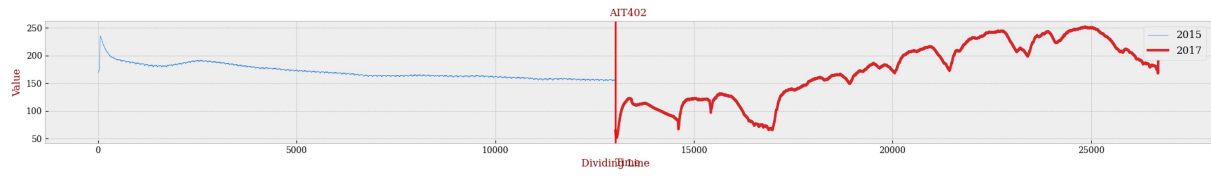
We compared the time efficiency of CDDIA with five baseline methods on three datasets, as shown in Fig. 6 and Table V. The experimental results indicate that the Hoeffding Tree, due to its improvement based on decision trees, has a significant



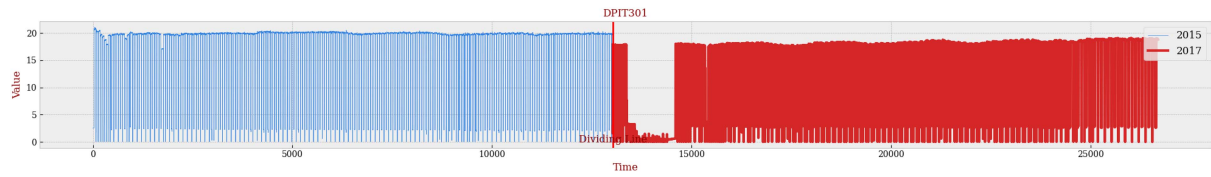
(a) Data collected by P402 in the years 2015 and 2017.



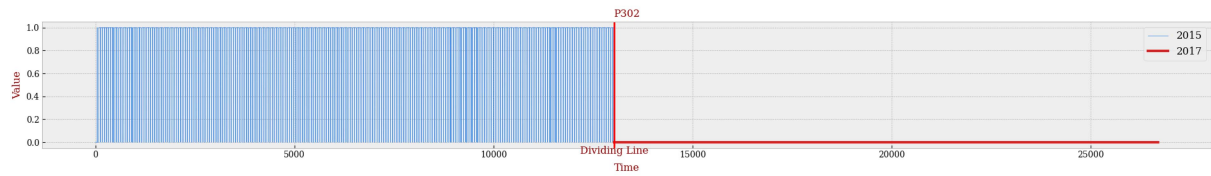
(b) Data collected by P401 in the years 2015 and 2017.



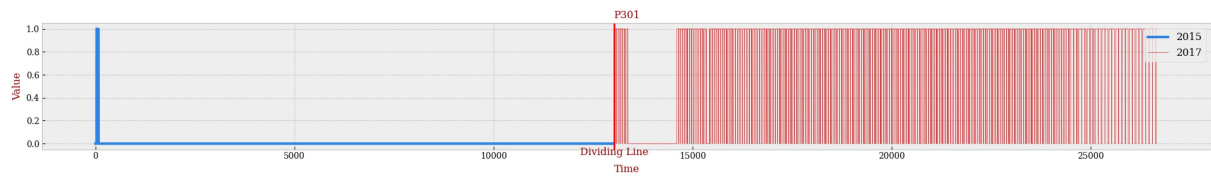
(c) Data collected by AIT402 in the years 2015 and 2017.



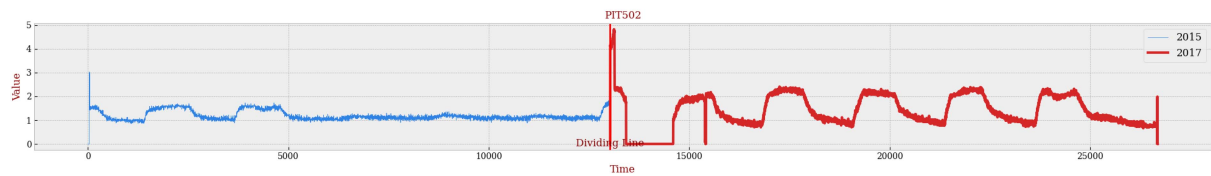
(d) Data collected by DPIT301 in the years 2015 and 2017.



(e) Data collected by P302 in the years 2015 and 2017.



(f) Data collected by P301 in the years 2015 and 2017.



(g) Data collected by PIT502 in the years 2015 and 2017.

Fig. 4. Seven images arranged as abcdefg.

TABLE III
INTRODUCTION TO SELECTED EQUIPMENT COMPONENTS

Name	Type	Function Description
P302	Actuator	Ultrafiltration Inlet Pump: Transports water from the ultrafiltration inlet tank through the ultrafiltration filtration pump to the reverse osmosis inlet tank
P301 (Backup)	Actuator	Backup Component for P302: Serves the same function as P302 and acts as a redundant unit
P402	Actuator	Pump: Transfers water from the reverse osmosis inlet tank to the ultraviolet dechlorinator
P401 (Backup)	Actuator	Backup Component for P402: Functions similarly to P402 and serves as a backup unit
AIT402	Sensor	ORP Meter: Controls NaHSO ₃ dosing (P203) and NaOCl dosing (P205)
DPIT301	Sensor	Differential Pressure Transmitter: Controls the backwashing process
PIT502	Sensor	Pressure Gauge: Monitors the pressure of the RO permeate

TABLE IV
COMPARISON OF SWAT (2015, 2017), BATADAL (2014, 2017) AND CICIDS2017 RESULTS (FOR TN, LARGER IS BETTER. FOR FP AND FPR, SMALLER VALUES IS BETTER)

Methods	SWaT			BATADAL			CICIDS2017		
	TN	FP	FPR	TN	FP	FPR	TN	FP	FPR
Streaming Random Patches (SRP) [38]	13661	0	0	1614	92	0.0539	20806	290	0.0137
HoeffdingTree [30]	13661	0	0	1647	59	0.0345	19695	1401	0.0664
BOLE [39]	13661	0	0	1686	20	0.0117	20688	408	0.0193
ARF-DDM [40]	13661	0	0	1699	7	0.0041	20801	295	0.0139
OWAD [25]	12197	1464	0.1072	1656	50	0.0293	20566	530	0.0251
CDDIA (ours)	13252↑	409↓	0.0299↓	1705↑	1↓	0.0005↓	20960↑	136↓	0.0064↓

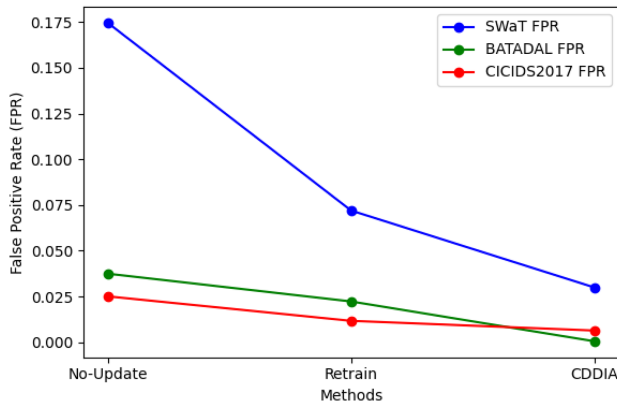


Fig. 5. Comparing our method with No-Update and Retrain, there is a very significant decrease in the false alarm rate.

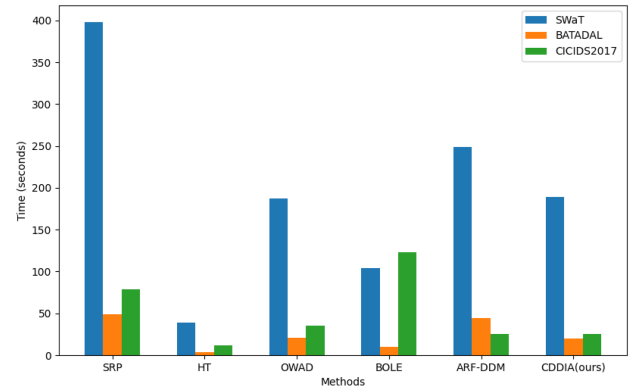


Fig. 6. Our method was compared with five baseline methods in terms of time efficiency.

TABLE V
ABLATION EXPERIMENTS: COMPARISON WITH NO-UPDATE AND RETRAIN

Methods	SWaT		BATADAL		CICIDS2017	
	FP	FPR	FP	FPR	FP	FPR
No-Update	2386	0.1747	64	0.0375	530	0.0251
Retrain	984	0.072	38	0.0223	246	0.0117
CDDIA (ours)	409↓	0.0299↓	1↓	0.0005↓	136↓	0.0064↓

advantage in efficiency, but its adaptability performance is weaker compared to other methods. CDDIA shows advantages on smaller datasets, achieving a balance between performance and efficiency. (Note that our method also includes the filtering of samples with drift at the sample level.)

VII. DISCUSSION AND FUTURE WORK

Indeed, our work addresses the significant issue of interpretability in industrial control system anomaly detection concerning concept drift, an aspect often overlooked in current concept drift techniques. In contrast to existing approaches that primarily focus on detecting changes in data distribution and adapting to new distributions, we emphasize understanding how concept drift affects the model, which is crucial given the tightly coupled nature of industrial control systems. One advantage of CDDIA is its ability to select new samples indicative of concept drift and retain relevant old samples, thereby mitigating the impact of concept drift through model retraining. This comprehensive adaptation strategy significantly enhances model stability and reliability.

Although concept drift adaptation methods based on retraining exhibit excellent adaptability to concept drift, the time and resource costs associated with maintenance and retraining increase as the sample size grows. Considering the limited computational resources of most industrial control system field devices, it is challenging to support extensive computations. Therefore, in our future work, we will focus on improving the efficiency of concept drift adaptation. This can involve researching and designing methods for model compression and lightweight, aiming to reduce the number of model parameters and computational complexity, thereby lowering the resource consumption of interpretation and adaptation algorithms.

VIII. CONCLUSION

In this paper, we propose a comprehensive framework to address concept drift in anomaly detection, covering three key aspects: detection, interpretation, and adaptation. Our work focuses on providing concept drift detection for unsupervised anomaly detection models and addressing the interpretability shortcomings in current concept drift techniques. Specifically, we concentrate on accurately capturing instances of concept drift in normal data within an unsupervised setting. We employ a combination of a search optimization algorithm and the SHAP method to interpret drift samples, enabling a comprehensive interpretation of drift samples at both the sample and feature levels. By filtering new and old samples and retraining the anomaly detection model, we enhance the model's ability to adapt to concept drift, thereby reducing its impact on the system and minimizing false positives. Experimental results indicate that our approach effectively reduces false positive rates caused by concept drift in comparison to previous research. Furthermore, the feature-level interpretation offers insights for experts into the true reasons behind concept drift occurrences. This comprehensive approach holds promise for enhancing IoT security, ensuring systems can effectively handle challenges arising from concept drift.

REFERENCES

- [1] Y. Sun, T. Chen, Q. V. H. Nguyen, and H. Yin, "TinyAD: Memory-efficient anomaly detection for time series data in industrial IoT," *IEEE Trans. Ind. Informat.*, vol. 20, no. 1, pp. 824–834, Jan. 2024.
- [2] H. Xu, G. Pang, Y. Wang, and Y. Wang, "Deep isolation forest for anomaly detection," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12591–12604, Dec. 2023.
- [3] L. Xu, B. Wang, X. Wu, D. Zhao, L. Zhang, and Z. Wang, "Detecting semantic attack in SCADA system: A behavioral model based on secondary labeling of states-duration evolution graph," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 2, pp. 703–715, Mar./Apr. 2022.
- [4] D. Zhao, G. Xiao, Z. Wang, L. Wang, and L. Xu, "Minimum dominating set of multiplex networks: Definition, application, and identification," *IEEE Trans. Syst., Man, Cybern. Syst.*, vol. 51, no. 12, pp. 7823–7837, Dec. 2021.
- [5] J. Jang, E. Hwang, and S.-H. Park, "N-pad: Neighboring pixel-based industrial anomaly detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 4364–4373.
- [6] D. Zhao, L. Wang, Z. Wang, and G. Xiao, "Virus propagation and patch distribution in multiplex networks: Modeling, analysis, and optimal allocation," *IEEE Trans. Inf. Forensics Security*, vol. 14, no. 7, pp. 1755–1767, Jul. 2019.
- [7] J. C. Schlimmer and R. H. Granger, "Incremental learning from noisy data," *Mach. Learn.*, vol. 1, pp. 317–354, 1986.
- [8] G. Widmer and M. Kubat, "Learning in the presence of concept drift and hidden contexts," *Mach. Learn.*, vol. 23, pp. 69–101, 1996.
- [9] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: A review," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 12, pp. 2346–2363, Dec. 2019.
- [10] H. Yu, W. Liu, J. Lu, Y. Wen, X. Luo, and G. Zhang, "Detecting group concept drift from multiple data streams," *Pattern Recognit.*, vol. 134, 2023, Art. no. 109113.
- [11] N. Lu, J. Lu, G. Zhang, and R. L. De Mantaras, "A concept drift-tolerant case-base editing technique," *Artif. Intell.*, vol. 230, pp. 108–133, 2016.
- [12] A. Liu, Y. Song, G. Zhang, and J. Lu, "Regional concept drift detection and density synchronized drift adaptation," in *Proc. Int. Joint Conf. Artif. Intell.*, 2017, pp. 2280–2286.
- [13] B. Krawczyk, L. L. Minku, J. Gama, J. Stefanowski, and M. Woźniak, "Ensemble learning for data stream analysis: A survey," *Inf. Fusion*, vol. 37, pp. 132–156, 2017.
- [14] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, "Learning with drift detection," in *Proc. 17th Braz. Symp. Artif. Intell.*, Sao Luis, Maranhao, Brazil, Springer, 2004, pp. 286–295.
- [15] M. Baena-Garcia, J. del Campo-Ávila, R. Fidalgo, A. Bifet, R. Gavalda, and R. Morales-Bueno, "Early drift detection method," in *Proc. 4th Int. Workshop Knowl. Discov. Data Streams*, 2006, pp. 77–86.
- [16] G. J. Ross, N. M. Adams, D. K. Tasoulis, and D. J. Hand, "Exponentially weighted moving average charts for detecting concept drift," *Pattern Recognit. Lett.*, vol. 33, no. 2, pp. 191–198, 2012.
- [17] R. S. Barros, D. R. Cabral, P. M. Gonçalves Jr, and S. G. Santos, "RDDM: Reactive drift detection method," *Expert Syst. Appl.*, vol. 90, pp. 344–355, 2017.
- [18] A. Liu, G. Zhang, and J. Lu, "Fuzzy time windowing for gradual concept drift adaptation," in *Proc. IEEE Int. Conf. Fuzzy Syst.*, 2017, pp. 1–6.
- [19] I. Frias-Blanco, J. del Campo-Ávila, G. Ramos-Jimenez, R. Morales-Bueno, A. Ortiz-Díaz, and Y. Caballero-Mota, "Online and non-parametric drift detection methods based on hoeffding's bounds," *IEEE Trans. Knowl. Data Eng.*, vol. 27, no. 3, pp. 810–823, Mar. 2015.
- [20] A. Bifet and R. Gavalda, "Learning from time-changing data with adaptive windowing," in *Proc. SIAM Int. Conf. Data Mining*, SIAM, 2007, pp. 443–448.
- [21] Ö. Gözümçik and F. Can, "Concept learning using one-class classifiers for implicit drift detection in evolving data streams," *Artif. Intell. Rev.*, vol. 54, pp. 3725–3747, 2021.
- [22] S. H. Bach and M. A. Maloof, "Paired learners for concept drift," in *Proc. 8th IEEE Int. Conf. Data Mining*, 2008, pp. 23–32.
- [23] T. Dasu, S. Krishnan, S. Venkatasubramanian, and K. Yi, "An information-theoretic approach to detecting changes in multi-dimensional data streams," in *Proc. Symp. Interface Statist. Comput. Sci. Appl.*, 2006.
- [24] L. Yang et al., "CADE: Detecting and explaining concept drift samples for security applications," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 2327–2344.
- [25] D. Han et al., "Anomaly detection in the open world: Normality shift detection, explanation, and adaptation," in *Proc. 30th Annu. Netw. Distrib. Syst. Secur. Symp.*, 2023.
- [26] P. Panda, V. N. Balasubramanian, and G. Sinha, "Feature-interpretable real concept drift detection," in *Proc. ICLR Workshop Pitfalls Limited Data Comput. Trustworthy ML*, 2023.

- [27] A. Bifet, G. Holmes, and B. Pfahringer, "Leveraging bagging for evolving data streams," in *Proc. Eur. Conf. Mach. Learn. Knowl. Discov. Databases*, Barcelona, Spain, Springer, 2010, pp. 135–150.
- [28] F. Chu and C. Zaniolo, "Fast and light boosting for adaptive mining of data streams," in *Proc. 8th Pacific-Asia Conf. Adv. Knowl. Discov. Data Mining*, Sydney, Australia, Springer, 2004, pp. 282–292.
- [29] H. M. Gomes et al., "Adaptive random forests for evolving data stream classification," *Mach. Learn.*, vol. 106, pp. 1469–1495, 2017.
- [30] G. Hulten, L. Spencer, and P. Domingos, "Mining time-changing data streams," in *Proc. 7th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2001, pp. 97–106.
- [31] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 4768–4777.
- [32] A. P. Mathur and N. O. Tippenhauer, "SWaT: A water treatment testbed for research and training on ics security," in *Proc. Int. Workshop Cyber-Phys. Syst. Smart Water Netw.*, 2016, pp. 31–36.
- [33] R. Taormina, S. Galelli, N. O. Tippenhauer, E. Salomons, and A. Ostfeld, "Characterizing cyber-physical attacks on water distribution systems," *J. Water Resour. Plan. Manage.*, vol. 143, no. 5, 2017, Art. no. 04017009.
- [34] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy*, 2018, pp. 108–116.
- [35] C. M. Ahmed, V. R. Palleti, and A. P. Mathur, "WADI: A water distribution testbed for research in the design of secure cyber physical systems," in *Proc. 3rd Int. Workshop Cyber-Phys. Syst. Smart Water Netw.*, 2017, pp. 25–28.
- [36] L. Xu, X. Ding, H. Peng, D. Zhao, and X. Li, "ADTCD: An adaptive anomaly detection approach towards concept-drift in IoT," *IEEE Internet of Things J.*, vol. 10, no. 18, pp. 15931–15942, Sep. 2023.
- [37] C. Tang, L. Xu, B. Yang, Y. Tang, and D. Zhao, "GRU-based interpretable multivariate time series anomaly detection in industrial control system," *Comput. Secur.*, vol. 127, 2023, Art. no. 103094, doi: [10.1016/j.cose.2023.103094](https://doi.org/10.1016/j.cose.2023.103094).
- [38] H. M. Gomes, J. Read, and A. Bifet, "Streaming random patches for evolving data stream classification," in *Proc. IEEE Int. Conf. Data Mining*, 2019, pp. 240–249.
- [39] R. S. M. de Barros, S. G. T. de Carvalho Santos, and P. M. G. Júnior, "A boosting-like online learning ensemble," in *Proc. Int. Joint Conf. Neural Netw.*, 2016, pp. 1871–1878.
- [40] A. Bifet, G. Holmes, B. Pfahringer, R. Kirkby, and R. Gavalda, "New ensemble methods for evolving data streams," in *Proc. 15th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2009, pp. 139–148.



Lijuan Xu received the PhD degree from the School of Cyber Security, Harbin Institute of Technology, in 2023. She is currently an associate professor with Shandong Computer Science Center (National Supercomputer Center in Jinan), China. Her main research interests include network security, industrial internet security, and computer forensics.



Ziyu Han received the bachelor's degree from the Zhengzhou University of Light Industry, in 2020, and is currently working toward the master's degree with the Qilu University of Technology. His main research interests include interpretability of anomaly detection, industrial internet security.



Dawei Zhao received the PhD degree in cryptology from the Beijing University of Posts and Telecommunications, in 2014. He is currently a professor with the Shandong Computer Science Center (National Supercomputer Center in Jinan), China. His main research interests include network security, complex network and epidemic spreading dynamics.



Xin Li received the PhD degree in cyberspace security from the Beijing University of Posts and Telecommunications, in 2022. He is currently an lecturer with the Shandong Computer Science Center (National Supercomputer Center in Jinan), China. His main research interests include software security, industrial internet security and machine learning.



Fuqiang Yu received the BSc and PhD degrees in computer science from Shandong University, Jinan, China, in 2017 and 2023, respectively. He is currently working with Shandong Provincial Key Laboratory of Computer Networks, Shandong Computer Science Center (National Supercomputer Center in Jinan), Qilu University of Technology (Shandong Academy of Sciences), Jinan, China. His research interests include deep learning, medical data mining, and geospatial applications of information technologies. He has been invited as reviewer for top journals, such as, *IEEE Transactions on Services Computing* and *IEEE Transactions on Knowledge and Data Engineering*. Also, he has been invited as external reviewer for conferences such as SIGKDD and ICDE.



Chuan Chen received the BS degree in mathematics from Henan University, Kaifeng, China, in 2005, the MS degree in mathematics from Capital Normal University, Beijing, China, in 2008, and the PhD degree in cryptography from the Beijing University of Posts and Telecommunications, Beijing, China, in 2019. He is currently working with the Qilu University of Technology (Shandong Academy of Sciences), Jinan, China. His current research interests include cyberspace security and the dynamic analysis of non-linear systems.